

Лабораторна робота №8

Тема: «Ресурси Keras. TensorFlow. Навчання лінійної регресії»

Мета: Дослідження ресурсу Keras і TensorFlow. Застосування TensorFlow.

Завдання 1 : Використовуючи засоби TensorFlow, реалізувати код наведений нижче та дослідити структуру розрахункового алгоритму. Для виконання розрахунків, можна використовувати онлайн – середовище google – colab (перехід за посиланням:

План роботи

1) спочатку створюємо випадковим чином вхідні дані X_data та y_data за таким алгоритмом: • створюємо 1000 випадкових точок рівномірно на інтервалі $[0; 1]$; • підраховуємо для кожної точки x відповідну «правильну відповідь» y за формулою $y = 2x + 1 + \epsilon$, де ϵ - випадково розподілений шум із дисперсією 2, $\epsilon \sim N(\epsilon; 0, 2)$;

2) потім оголошуємо `tf.placeholder` для змінних X та y ; на цьому етапі вже потрібно задати їм розмірність, і це в нашому випадку матриця розмірності (розмір міні-батча $\times 1$) для X і просто вектор довжини розмір міні-батча для y ;

3) далі ініціалізуємо змінні k та b ; це змінні TensorFlow, які поки що жодних значень не мають, але будуть ініціалізовані стандартним нормальним розподілом для k і нулем для b ;

4) потім ми встановлюємо власне суть моделі і при цьому будуємо функцію помилки $\sum - 2 (\hat{y})$ і y ; зверніть увагу на функцію `reduce_sum`: на виході вона лише підраховує суму матриці по рядках, але користуватися треба саме нею, а не звичайною сумою або відповідними функціями з `numpy`, тому що так TensorFlow зможе куди більш ефективно оптимізувати процес обчислень;

5) вводимо змінну `optimizer` - оптимізатор, тобто власне алгоритм, який підраховуватиме градієнти та оновлюватиме ваги; ми вибрали стандартний оптимізатор стохастичного градієнтного спуску; Тепер нам важливо лише відзначити, що тепер щоразу, коли ми просимо TensorFlow підрахувати значення змінної `optimizer`, десь за лаштунками відбуватимуться оновлення змінних, від

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.25.125.12.000 –Лр.8		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Лук'яненко В. Я.			Звіт з лабораторної роботи №8	Лім.	Арк.
Перевір.		Маєвський О. В.					1
Реценз.						ФІКТ, гр. КБ-23-1	
Н. Контр.							
Зав.каф.		Єфіменко А.А.					

яких залежить оптимізована змінна loss, тобто k і b ; по X та y оптимізації не буде, тому що значення `tf.placeholder` повинні бути жорстко задані, це вхідні дані;

б) записуємо великий цикл, що робить ці оновлення (тобто багато разів обчислює змінну `optimizer`); на кожній ітерації циклу ми беремо випадкове підмножина з `batch_size` (тобто 100) індексів даних та підраховуємо значення потрібних змінних; ми подаємо в функцію `sess.run` список змінних, які потрібно підрахувати (головне - "обчислити" змінну `optimizer`, інші потрібні тільки для виводу налагодження), і словник `feed_dict`, в який записуємо значення вхідних змінних, позначених раніше як `tf.placeholder`.

Лістинг код :

```
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"

import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

tf.compat.v1.disable_eager_execution()

np.random.seed(42)
tf.compat.v1.set_random_seed(42)

n_samples = 1000
batch_size = 100
num_steps = 20000

X_data = np.random.uniform(0, 1, (n_samples, 1)).astype(np.float32)
y_data = 2 * X_data + 1 + np.random.normal(0, 0.2, (n_samples, 1)).astype(np.float32)

X = tf.compat.v1.placeholder(tf.float32, shape=(batch_size, 1))
y = tf.compat.v1.placeholder(tf.float32, shape=(batch_size, 1))

with tf.compat.v1.variable_scope("linear_regression"):
    k = tf.Variable(tf.random.normal([1, 1], stddev=0.1), name="slope")
    b = tf.Variable(tf.zeros([1,]), name="bias")

y_pred = tf.matmul(X, k) + b

loss = tf.reduce_mean(tf.square(y - y_pred))

optimizer = tf.compat.v1.train.GradientDescentOptimizer(
    learning_rate=0.05
).minimize(loss)

display_step = 1000

with tf.compat.v1.Session() as sess:
    sess.run(tf.compat.v1.global_variables_initializer())
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.8	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		2

```

for step in range(num_steps):
    indices = np.random.choice(n_samples, batch_size, replace=False)
    X_batch = X_data[indices]
    y_batch = y_data[indices]

    _, loss_val, k_val, b_val = sess.run(
        [optimizer, loss, k, b],
        feed_dict={X: X_batch, y: y_batch}
    )

    if (step + 1) % display_step == 0:
        print(
            f"Епоха {step + 1}: loss = {loss_val:.6f}, "
            f"k = {k_val[0][0]:.4f}, b = {b_val[0]:.4f}"
        )

    final_k, final_b = sess.run([k, b])

print("\nФінальні параметри:")
print(f"k = {final_k[0][0]:.4f}")
print(f"b = {final_b[0]:.4f}")

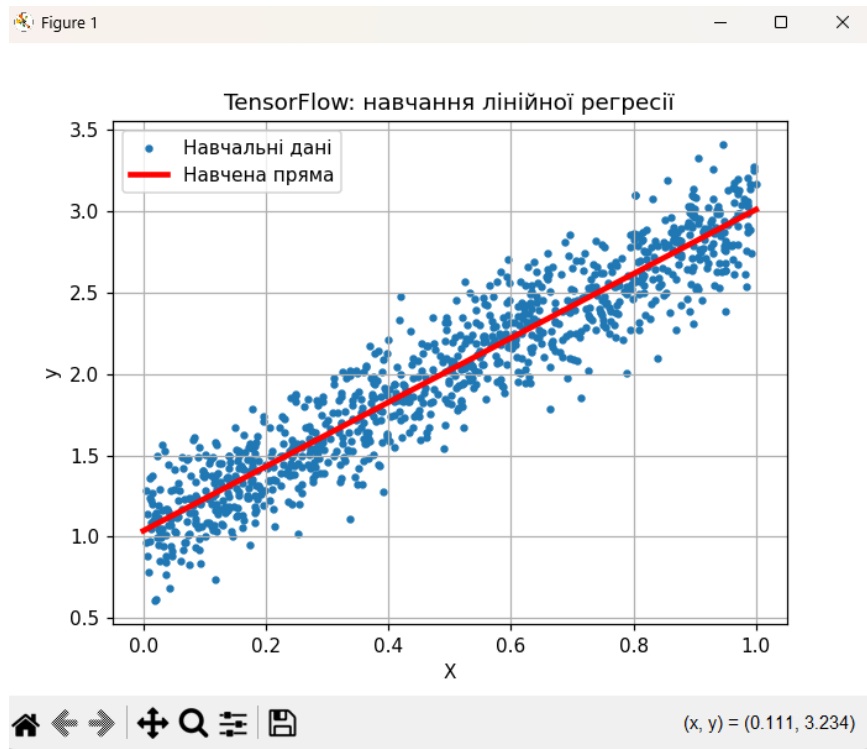
X_line = np.linspace(0, 1, 100).reshape(-1, 1)
y_line = final_k[0][0] * X_line + final_b[0]

plt.scatter(X_data, y_data, s=10, label="Навчальні дані")
plt.plot(X_line, y_line, color="red", linewidth=3, label="Навчена пряма")
plt.xlabel("X")
plt.ylabel("y")
plt.title("TensorFlow: навчання лінійної регресії")
plt.legend()
plt.grid(True)
plt.show()

```

Результат виконання коду.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.24.125.12.000 –Лр.8	Арк.
						3
Змн.	Арк.	№ докум.	Підпис	Дата		



```

Епоха 10000: loss = 0.037716, k = 1.9676, b = 1.0331
Епоха 11000: loss = 0.041487, k = 1.9633, b = 1.0323
Епоха 12000: loss = 0.039300, k = 1.9669, b = 1.0414
Епоха 13000: loss = 0.044595, k = 1.9590, b = 1.0302
Епоха 14000: loss = 0.035925, k = 1.9691, b = 1.0380
Епоха 15000: loss = 0.028687, k = 1.9669, b = 1.0359
Епоха 16000: loss = 0.041710, k = 1.9615, b = 1.0413
Епоха 17000: loss = 0.035020, k = 1.9662, b = 1.0412
Епоха 18000: loss = 0.043787, k = 1.9697, b = 1.0346
Епоха 19000: loss = 0.043563, k = 1.9726, b = 1.0389
Епоха 20000: loss = 0.039102, k = 1.9700, b = 1.0331

Фінальні параметри:
k = 1.9721
b = 1.0372

```

Висновок : У лабораторній роботі було досліджено використання TensorFlow для навчання моделі лінійної регресії. Було згенеровано випадкові навчальні дані за формулою $y = 2x + 1 + \text{шум}$. Параметри моделі k та b навчалися за допомогою градієнтного спуску шляхом мінімізації суми квадратів похибок. У результаті навчання значення k наблизилося до 2, а b — до 1, що підтверджує правильність роботи алгоритму.

Посилання на [GitHub](#)